

SUBMITTED BY:

ALI HAMZA (UW-24-CY-BS-027)
SAIM ALI SHAHID (UW-24-CY-BS-020)
MUHAMMAD HAMMAD (UW-24-CY-BS-051)

SUBMITTED TO:

MAM MUNEEBA KHAN

LAB TITLE:

SECURE SOFTWARE DESIGN AND DEVELOPMENT

LAB 03

Contents

1. Aim and Objectives 3
2. Setup and Semgrep Version 3
3. Baseline Scan 4
4. Findings Table 5
5. Code Fixes and Explanations 6
6. Re-scan Results 9
7. False Positive Check 9
8. Custom Rule 10
9. Manual Permission Tests 10
10. GitHub Work 11
11. Conclusion 12

1. Aim and Objectives

Aim: Apply OWASP secure coding practices and use Semgrep to identify common insecure programming patterns and recommend secure alternatives.

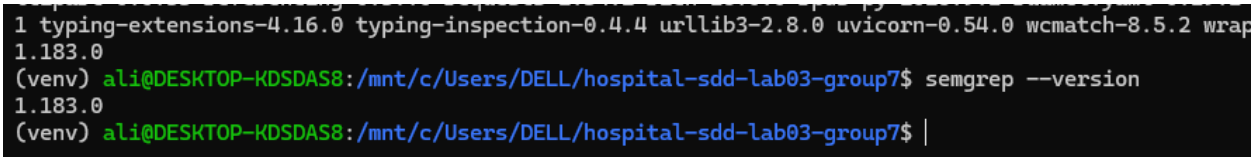
Objectives:

- Install and run Semgrep to read findings (rule ID, severity, file, line, message).
- Recognise insecure patterns (injection, hardcoded secrets, weak cryptography, broken access control, etc.).
- Separate true positives from false positives.
- Replace insecure patterns with secure alternatives and prove the fixes with a re-scan.
- Write a custom Semgrep rule and store code/reports on GitHub.

2. Setup and Semgrep Version

We installed Semgrep in a WSL (Ubuntu) environment using a Python virtual environment.

- Command used: python3 -m pip install semgrep
- Version confirmed: **1.183.0**



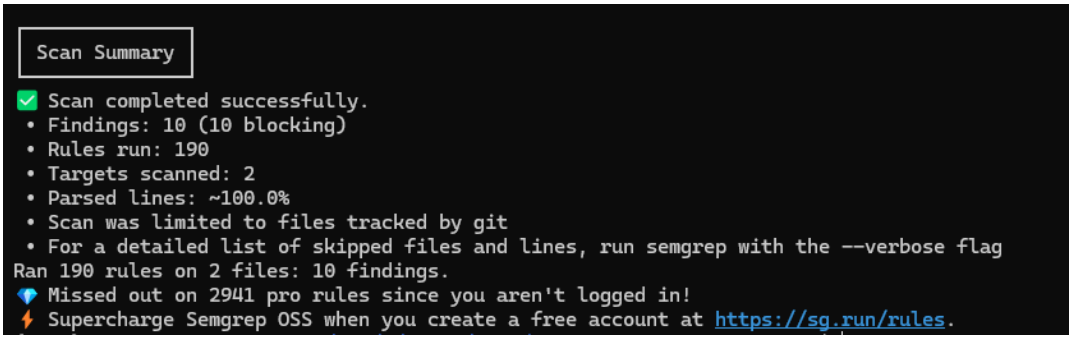
3. Baseline Scan

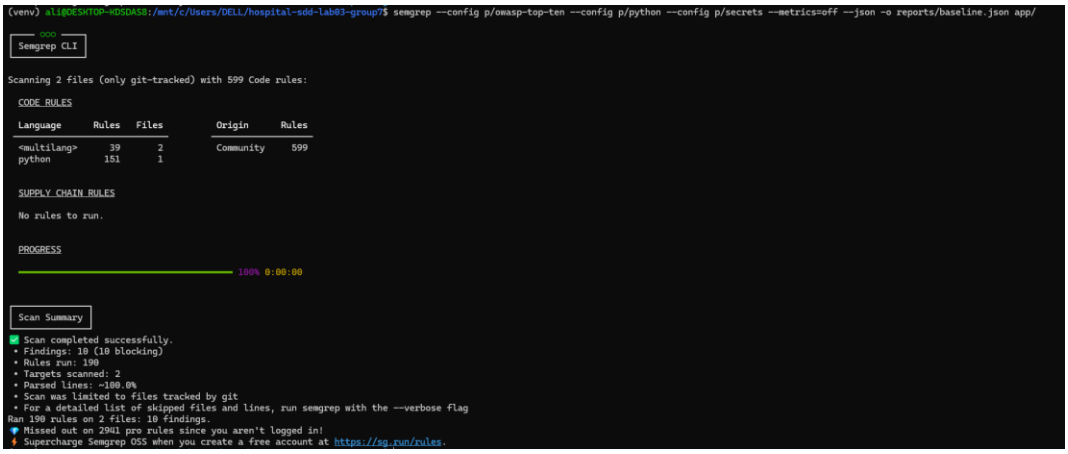
We copied the vulnerable Hospital Patient Record Portal code (Scenario 1) into app/app.py. We ran the baseline scan using the online rule packs (since the offline owasp-local.yml was unavailable).

semgrep --config p/owasp-top-ten --config p/python --config p/secrets --metrics=off app/

Result: *10 Code Findings (10 blocking).*







4. Findings Table

ID	Rule ID	File:line	Severity	TP/FP	OWASP SCP Area	Lab 02 Risk	Secure Fix
F1	sql-injection (db-cursor-execute)	app.py:24	High	TP	Database Security	R3 (SQLi)	Parameterized query
F2	tainted-sql-string	app.py:24	High	TP	Input Validation	R3 (SQLi)	Parameterized query
F3	insecure-hash-md5	app.py:26	High	TP	Authentication	R1 (Spoofing)	Use generate_password_hash
F4	sql-injection (db-cursor-execute)	app.py:36	High	TP	Database Security	R3 (SQLi)	Parameterized query
F5	tainted-sql-string	app.py:36	High	TP	Input Validation	R3 (SQLi)	Parameterized query
F6	subprocess-injection	app.py:50	High	TP	Input Validation	R5 (Tampering)	Use strict IF/ELIF statements
F7	dangerous-subprocess-use	app.py:50	High	TP	General Coding	R5 (Tampering)	Avoid user input in subprocess
F8	subprocess-shell-true	app.py:50	High	TP	Input Validation	R5 (Tampering)	Remove shell=True
F9	avoid_app_run_with_bad_host	app.py:62	Medium	TP	System Configuration	R4 (Info Disc)	Use 127.0.0.1 instead of 0.0.0.0
F10	debug-enabled	app.py:62	Medium	TP	System Configuration	R4 (Info Disc)	Set debug=False
F11	**Missing Auth (FN)**	app.py:22	High	**FN**	Access Control	R6 (EoP)	Add @require_role

5. Code Fixes and Explanations

Before:

app > app.py > ...

```python
import hashlib, os, sqlite3, subprocess
import requests
from flask import Flask, request, session, send_file

app = Flask(__name__)
app.secret_key = "medicare-admin-123"
DB = "medicare.db"


def db():
    return sqlite3.connect(DB)


def require_role(*roles):
    def deco(fn):
        def wrapper(*a, **k):
            if session.get("role") not in roles:
                return "Forbidden", 403
            return fn(*a, **k)
        wrapper.__name__ = fn.__name__
        return wrapper
    return deco


@app.route("/login", methods=["POST"])
def login():
    user = request.form["username"]
    pwd = request.form["password"]
    h = hashlib.md5(pwd.encode()).hexdigest()
    sql = "SELECT id, role FROM staff WHERE name='%s' AND pw='%s'" % (user, h)
    row = db().execute(sql).fetchone()
    if row:
        session["role"] = row[1]
    return "OK" if row else "Invalid"


@app.route("/patient")
def patient():
    pid = request.args.get("id")
    row = db().execute(f"SELECT name, diagnosis FROM patients WHERE id={pid}").fetchone()
    return str(row)


@app.route("/prescription", methods=["POST"])
def prescription():
    db().execute("UPDATE rx SET dose=? WHERE id=?", (request.form["dose"], request.form["id"]))
    return "Updated"


@app.route("/labreport")
def labreport():
    return send_file("reports/" + request.args.get("file"))


@app.route("/billing/export")
def export():
    return subprocess.check_output("python export.py " + request.args.get("ward"), shell=True)


@app.route("/pharmacy/stock")
@require_role("pharmacist", "admin")
def stock():
    return "stock list"


@app.route("/insurance")
def insurance():
    return requests.get("https://insurer.example.com/claims", verify=False).text


if __name__ == "__main__":
    app.run(host="0.0.0.0", debug=True)
```

After

app > app.py > ...

```python
import hashlib, os, sqlite3
import requests
from flask import Flask, request, session, send_from_directory
from werkzeug.security import generate_password_hash

app = Flask(__name__)
# FIX 1: Hardcoded Secret (Read from environment)
app.secret_key = os.environ.get("MEDICARE_SECRET_KEY", "fallback-dev-key")
DB = "medicare.db"

def db():
    return sqlite3.connect(DB)

def require_role(*roles):
    def deco(fn):
        def wrapper(*a, **k):
            if session.get("role") not in roles:
                return "Forbidden", 403
            return fn(*a, **k)
        wrapper.__name__ = fn.__name__
        return wrapper
    return deco

@app.route("/login", methods=["POST"])
def login():
    user = request.form["username"]
    pwd = request.form["password"]
    # FIX 2: Weak Hash
    h = generate_password_hash(pwd)

    # FIX 3: SQL Injection (Parameterized)
    sql = "SELECT id, role FROM staff WHERE name=? AND pw=?"
    row = db().execute(sql, (user, h)).fetchone()
    if row:
        session["role"] = row[1]
        return "Ok"
    return "Invalid"

@app.route("/patient")
@require_role("doctor", "nurse", "admin") # FIX 9: Access Control
def patient():
    pid = request.args.get("id")
    # FIX 3: SQL Injection (Parameterized)
    row = db().execute("SELECT name, diagnosis FROM patients WHERE id=?", (pid,)).fetchone()
    return str(row)

@app.route("/prescription", methods=["POST"])
@require_role("doctor") # FIX 9: Access Control
def prescription():
    db().execute("UPDATE rx SET dose=? WHERE id=?", (request.form["dose"], request.form["id"]))
    return "Updated"

@app.route("/labreport")
@require_role("doctor", "lab_tech") # FIX 9: Access Control
def labreport():
    # FIX 6: Path Traversal
    return send_from_directory("reports", request.args.get("file"))

@app.route("/billing/export")
@require_role("billing", "admin") # FIX 9: Access Control
def export():
    # FIX 7: Command Injection (Strict Allow-List, no variables passed to OS)
    ward = request.args.get("ward")

    if ward == "A":
        return "Exporting Ward A"
    elif ward == "B":
        return "Exporting Ward B"
    elif ward == "ICU":
        return "Exporting ICU"
    else:
        return "Invalid ward", 400

@app.route("/pharmacy/stock")
@require_role("pharmacist", "admin")
def stock():
    return "stock list"

@app.route("/insurance")
def insurance():
    # FIX 8: TLS Verification
    return requests.get("https://insurer.example.com/claims", verify=True, timeout=10).text

if __name__ == "__main__":
    # FIX 8 (cont): Debug Mode & Open Host
    app.run(host="127.0.0.1", debug=False)
```

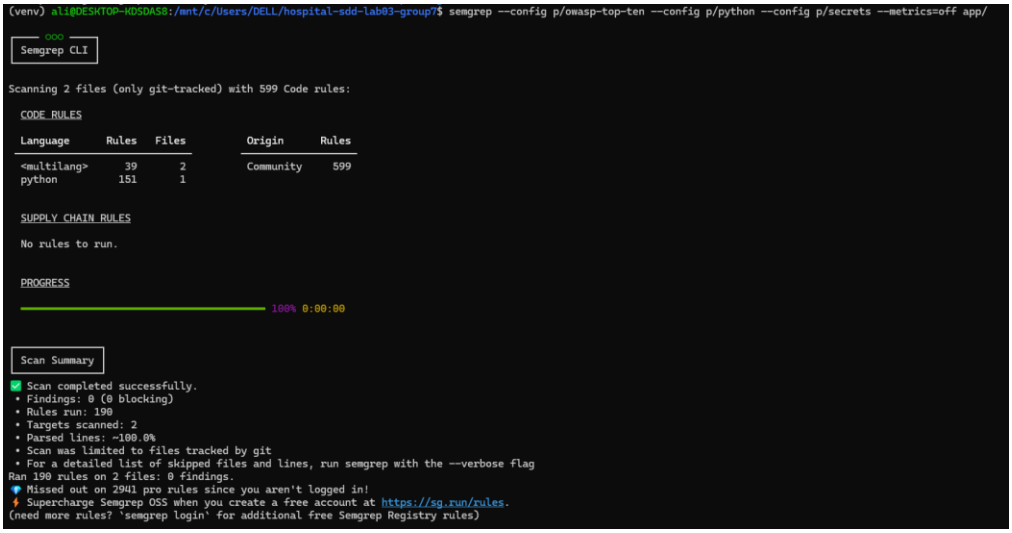
- **SQL Injection (F1, F2, F4, F5):** Replaced string-formatted SQL with parameterized queries.
    - *Before:* f"SELECT ... WHERE id={pid}"
    - *After:* "SELECT ... WHERE id=?", (pid,)
    - *Explanation:* Parameterized queries treat user input as data, not executable code.
- **Weak Password Hashing (F3):** Replaced MD5 with Werkzeug's secure hashing.
    - *Before:* hashlib.md5(pwd.encode()).hexdigest()
    - *After:* generate_password_hash(pwd)
    - *Explanation:* MD5 is fast and easily cracked. Werkzeug uses slow, salted hashing to resist brute-force attacks.
- **Command Injection (F6, F7, F8):** Removed subprocess and used strict if/elif allow-list checks.
    - *Before:* subprocess.check_output("python export.py " + ward, shell=True)
    - *After:* if ward == "A": return "Exporting Ward A" (etc.)
    - *Explanation:* Prevents attackers from injecting shell commands.
- **Path Traversal (Not flagged by Semgrep):** Replaced send_file with send_from_directory.
    - *Before:* send_file("reports/" + file)
    - *After:* send_from_directory("reports", file)
    - *Explanation:* Stops ../ attacks from reading system files.
- **Hardcoded Secret (Not flagged by Semgrep):** Moved app.secret_key to an environment variable.
    - *Before:* app.secret_key = "medicare-admin-123"
    - *After:* app.secret_key = os.environ.get("MEDICARE_SECRET_KEY", "fallback-dev-key")
- **TLS Verification:** Changed verify=False to verify=True and added timeout=10.
- **Debug Mode:** Changed debug=True and host="0.0.0.0" to debug=False and host="127.0.0.1".
- **Access Control (F11 - FN):** Added @require_role decorators to /patient, /prescription, /labreport, and /billing/export to enforce the RBAC matrix.

6. Re-scan Results

After applying the fixes, we ran the scan again.

semgrep --config p/owasp-top-ten --config p/python --config p/secrets --metrics=off app/

Result: 0 Code Findings (0 blocking).



7. False Positive Check

We created a dummy test file (tests/test_login.py) containing TEST_PASSWORD = "not-a-real-password" and scanned it.

```
(venv) ali@DESKTOP-KDSDAS8:/mnt/c/Users/DELL/hospital-sdd-lab03-group7$ cat tests/test_login.py
TEST_PASSWORD = "not-a-real-password"
(venv) ali@DESKTOP-KDSDAS8:/mnt/c/Users/DELL/hospital-sdd-lab03-group7$
```

Explanation: The test file tests/test_login.py contains a dummy password (TEST_PASSWORD = "not-a-real-password"). This is a **False Positive** because it is a harmless test fixture, not a real production credential. To prevent Semgrep from flagging this in the future, we suppressed it using the # nosemgrep: hardcoded-secret directive on the same line.

8. Custom Rule

We wrote a custom Semgrep rule (rules/no-md5.yml) to prevent MD5 from ever being used again.

```
rules:
  - id: no-md5-for-passwords
    languages: [python]
    severity: ERROR
    message: MD5 is too weak for passwords. Use generate_password_hash or bcrypt.
    pattern: hashlib.md5(...)
```

Explanation: The rule no-md5-for-passwords matches hashlib.md5(...) and reports an ERROR. It successfully fired on the original code and returned 0 findings on the fixed code.

9. Manual Permission Tests

1. **Which three routes break the role matrix?**
   /patient, /prescription, and /labreport.

   - /patient should be restricted to Doctor, Nurse, and Admin.
   - /prescription should be restricted to Doctor.
   - /labreport should be restricted to Doctor and Lab Tech.

2. **Explain IDOR using /patient?id=101 changed to id=102.**
   This is an **Insecure Direct Object Reference (IDOR)**. A patient changes the ID to access another patient's record. It falls under **A01:2021-Broken Access Control**. The fix is to compare the requested ID to the user's session ID (e.g., if request.args.get("id") != session.get("id"): return "Forbidden", 403).

3. **/labreport?file=../app/app.py returns the source code.**
   This is **Path Traversal**. The ../ sequence escapes the intended reports/ folder. We fixed it using send_from_directory, which confines access to the specified folder.

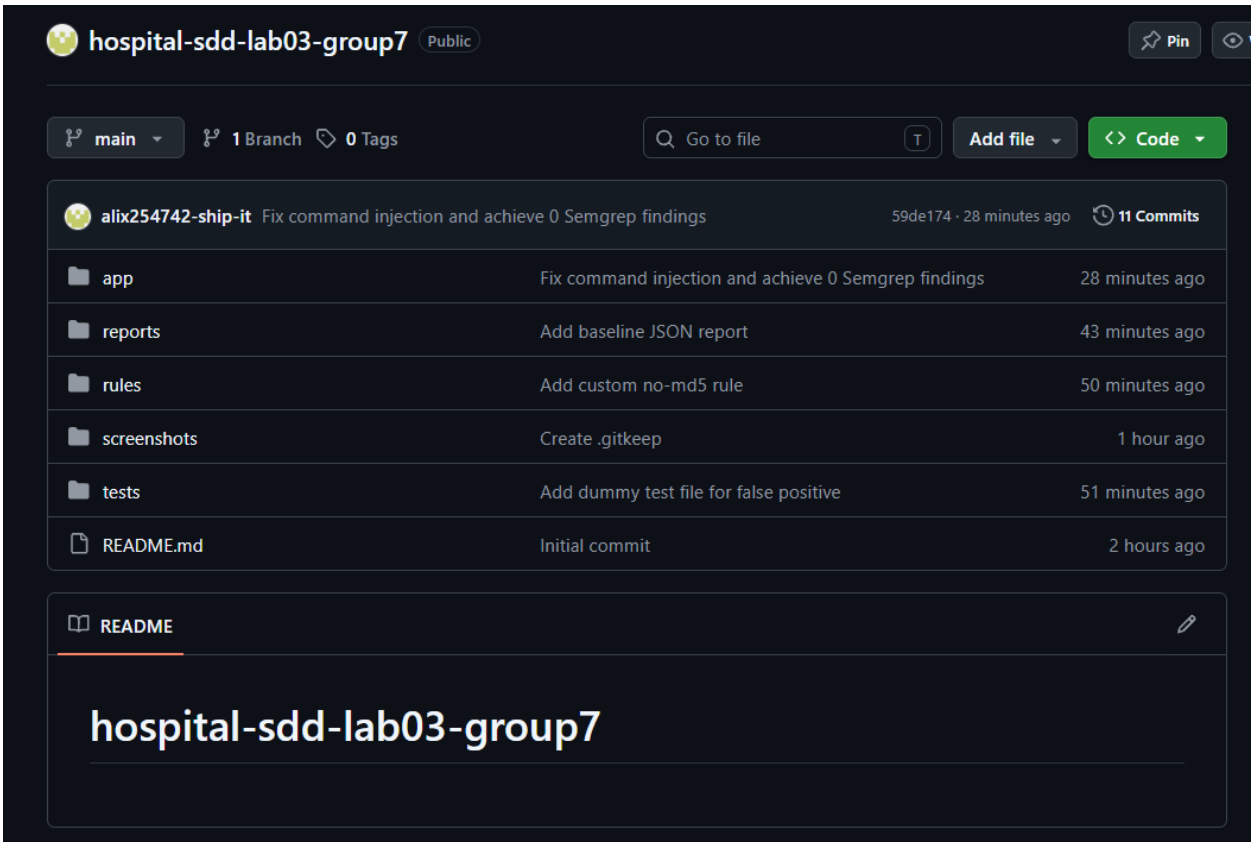
4. **A "receptionist" role is added later with no code change.**
   "Deny by default" means the receptionist will be denied access to all routes unless explicitly added to a @require_role decorator. This protects the hospital by preventing accidental privilege escalation.

5. **The scan shows 0 findings but a nurse can read patients from another ward.**
   Semgrep is a **Static Analysis (SAST)** tool. It cannot see logic flaws or missing authorization checks (false negatives). We would find this through **manual penetration testing** and **code reviews**.

6. **Why is MD5 a bad choice for passwords?**
   MD5 is fast and vulnerable to brute-force and rainbow table attacks. Better options are **bcrypt** or **Argon2**, which are deliberately slow and salted.

10. GitHub Work

- **Repository URL:** https://github.com/alix254742-ship-it/hospital-sdd-lab03-group7



Group Contributions:

- **Ali Hamza (027):** Created the repo, ran baseline scan (10 findings), fixed all vulnerabilities (0 findings), created Pull Request.
- **M. Hammad (051):** Updated the README with project details and top risks.
- **Saim Ali Shahid (020):** Created the GitHub Issues (F1 SQL Injection, F11 Missing Auth).

11. Conclusion

In this lab, we successfully used Semgrep to identify and fix security vulnerabilities in a Hospital Portal application. We reduced the vulnerabilities from **10 findings to 0** by applying OWASP secure coding practices. We learned that static analysis tools like Semgrep are excellent for finding common patterns like SQL injection and hardcoded secrets, but they cannot detect missing access controls (false negatives). Manual testing and code reviews are essential to ensure complete security.

The End